

Lab Introduction to Query

Name: Lê Hồng Anh

Student ID: 11247126

Lab Exercise

Lab Exercise 1: Triggering Nested Loop Join

Result:

Query

Query History

1

EXPLAIN (ANALYZE, BUFFERS)

2

SELECT c.first_name, c.last_name, r.rental_date

3

FROM customer c

4

JOIN rental r ON c.customer_id = r.customer_id

5

WHERE c.customer_id = 1;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 14

✎

QUERY PLAN

text

🔒

1

Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=10.919..26.206 rows=32.00 loops=1)

2

Buffers: shared hit=1 read=152

3

-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.057..0.060 rows=1.00 loo...

4

Index Cond: (customer_id = 1)

5

Index Searches: 1

6

Buffers: shared hit=1 read=2

7

-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=10.858..26.131 rows=32.00 loops=1)

8

Filter: (customer_id = 1)

9

Rows Removed by Filter: 16012

10

Buffers: shared read=150

11

Planning:

12

Buffers: shared hit=96 read=5

13

Planning Time: 72.207 ms

14

Execution Time: 26.284 ms

Discussion

1, Identify the Roles: Look at the plan tree. Which table is the Outer (top) and which is the Inner (bottom)?

- **Outer Table (Driving Table / Top):** `customer c`
 - **Explanation:** In the plan tree, PostgreSQL scans `customer c` first using an **Index Scan** (`customer_pkey`). It serves as the outer loop driving the join.
 - **Inner Table (Driven Table / Bottom):** `rental r`
 - **Explanation:** The inner child node is a **Seq Scan** on `rental r` with the filter (`customer_id = 1`). PostgreSQL evaluates this inner relation for the record fetched from the outer relation.
- 2, Why Nested Loop? How many rows did the customer table return? Why does this make Hash Join "too expensive"?
- **Rows returned by `customer`:** Exactly **1 row** (`rows=1.00`) because the predicate `c.customer_id = 1` performs a unique key lookup via `customer_pkey`.
 - **Why Hash Join is too expensive:**
 - A **Hash Join** requires significant startup overhead: allocating working memory (`work_mem`), building an in-memory hash table across an entire relation, and hashing the join keys.
 - Because the outer relation yields only **a single row**, running a single iteration in a **Nested Loop Join** incurs almost zero memory overhead and no hash-table setup cost. The planner recognizes that building a hash structure for just 1 matching key is far more expensive than simply streaming into the loop.
- 3, Efficiency: Look at loops in the Inner Table node. Why is it 1?
- **Why `loops = 1`:**
 - In a Nested Loop Join, the inner table scan is executed once for every row produced by the outer table.
 - Since the outer table (`customer`) returned exactly **1 row** (`rows=1.00`), the planner only had to execute the inner scan on `rental` **exactly once**. Hence, `loops=1`.

Notice that the inner scan used a `Seq Scan` instead of an `Index Scan`, scanning 150 disk blocks and filtering out 16,012 rows. Adding an index on `rental(customer_id)` would further optimize this step by replacing the sequential scan with an index seek.

Lab Exercise 2: Triggering Hash Join

Result

```
1  EXPLAIN (ANALYZE, BUFFERS)
2  SELECT f.title, l.name as language
3  FROM film f
4  JOIN language l ON f.language_id = l.language_id;
```

Data Output Messages Notifications

Showing rows

	QUERY PLAN
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=9.541..9.952 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=88 read=1
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.020..0.212 rows=1000.00 loo...
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=9.506..9.507 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=9.491..9.494 rows=6.00 loo...
10	Buffers: shared read=1
11	Planning:
12	Buffers: shared hit=36 read=1
13	Planning Time: 40.314 ms
14	Execution Time: 10.015 ms

If `work_mem = '64kB'`:

Query

Query History

```

1 SET work_mem = '64kB';
2
3 EXPLAIN (ANALYZE, BUFFERS)
4 SELECT f.title, l.name as language
5 FROM film f
6 JOIN language l ON f.language_id = l.language_id;
7
8 RESET work_mem;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing row

	QUERY PLAN	
	text	🔒
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.033..0.338 rows=1000.00 loops=1)	
2	Hash Cond: (f.language_id = l.language_id)	
3	Buffers: shared hit=89	
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.017..0.122 rows=1000.00 loo...	
5	Buffers: shared hit=88	
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.011..0.011 rows=6.00 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
8	Buffers: shared hit=1	
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.007..0.008 rows=6.00 loo...	
10	Buffers: shared hit=1	
11	Planning Time: 0.243 ms	
12	Execution Time: 0.392 ms	

Discussion

1, Identify Phases: Which table is being Hashed (Built)? Which table is being Probed?

- **Hashed (Built / Inner Table):** language l . It appears under the Hash node (Seq Scan on language l), where PostgreSQL scans all 6 rows and builds an in-memory hash table.
- **Probed (Outer Table):** film f . It is sequentially scanned (Seq Scan on film f), and each row's language_id is evaluated against the hash table to find matches.

2, Memory Check: Look at the Hash node. What is the Memory Usage?

- **Observed Memory:** Exactly 9kB (with Buckets: 1024 , Batches: 1).
 - **Meaning:** Batches: 1 confirms that the entire hash table fits inside RAM with zero spills to temporary files on disk.
- 3, Hypothesis: What happens if you run `SET work_mem = '64kB'` ; and run it again?
- **Observation:** The execution plan strategy remains completely identical (Hash Join , Batches: 1 , Memory Usage: 9kB).
 - **Why the plan did not change:** The hash table only requires 9kB , which is far below the reduced limit of 64kB . Thus, memory remains sufficient for an in-memory join without spilling.
 - **Why the execution time dropped (from ~10 ms to ~0.39 ms):** This improvement is caused by **Buffer Caching**, not the `work_mem` setting. The first run had to fetch cold blocks from disk (`read=1`), whereas the second run retrieved all data directly from the warm buffer pool in RAM (`shared hit=89` , `read=0`).

Lab Exercise 3: Triggering Merge Join

Result:

Query Query History

```
1 SET enable_hashjoin = off;
2 SET enable_nestloop = off;
3
4 EXPLAIN (ANALYZE, BUFFERS)
5 SELECT f.film_id, f.title, i.inventory_id
6 FROM film f
```

Data Output Messages Notifications

≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL

Showing rows: 1 to 16



QUERY PLAN

text



1	Merge Join (cost=349.64..547.91 rows=4581 width=23) (actual time=0.920..1.973 rows=4581.00 loops=1)
2	Merge Cond: (f.film_id = i.film_id)
3	Buffers: shared hit=109
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.008..0.219 rows=1000.00 loops=1)
5	Index Searches: 1
6	Buffers: shared hit=84
7	-> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=0.908..1.064 rows=4581.00 loops=1)
8	Sort Key: i.film_id
9	Sort Method: quicksort Memory: 300kB
10	Buffers: shared hit=25
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.011..0.388 rows=4581.00 loops=1)
12	Buffers: shared hit=25
13	Planning:
14	Buffers: shared hit=6
15	Planning Time: 0.223 ms
16	Execution Time: 2.172 ms

Total rows: 16 Query complete 00:00:00.072

after RESET ALL:

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
1 RESET ALL;  
2  
3 EXPLAIN (ANALYZE, BUFFERS)  
4 SELECT f.film_id, f.title, i.inventory_id  
5 FROM film f  
6 JOIN inventory i ON f.film_id = i.film_id;
```

Below the query, the 'Data Output' tab is active, displaying the 'QUERY PLAN' for the executed query. The plan shows a Hash Join operation with a Hash Cond: (i.film_id = f.film_id). The plan also shows the Buffers: shared hit=113. The plan is as follows:

Step	Operation
1	Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=0.500..1.603 rows=4581.00 loops=1)
2	Hash Cond: (i.film_id = f.film_id)
3	Buffers: shared hit=113
4	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.020..0.225 rows=4581.00 loops=1)
5	Buffers: shared hit=25
6	-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=0.473..0.473 rows=1000.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 59kB
8	Buffers: shared hit=88
9	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.009..0.230 rows=1000.00 loops=1)
10	Buffers: shared hit=88
11	Planning:
12	Buffers: shared hit=6
13	Planning Time: 0.299 ms
14	Execution Time: 1.759 ms

Discussion

1, Structure: Do you see a Sort node? Or did it use Index Scan to get sorted data for free?

- **Sort Node vs. Index Scan (Hybrid behavior):** Both techniques were utilized
 - **Table film f (The "Free Lunch"):** Postgres used an **Index Scan** via the primary key film_pkey (cost=0.28..127.33). Because B-Tree indexes store keys in sorted order, the data from film was retrieved pre-sorted by film_id at near-zero extra cost.

- **Table inventory i (The "Expensive Bill"):** Postgres performed a **Seq Scan** followed by an explicit **Sort** node (Sort Key: i.film_id , using quicksort Memory: 300kB with cost 349.37..360.82). This explicit sort was required because there was no pre-ordered index available for scanning inventory on film_id in this specific path.

2, Compare: Reset the settings (RESET ALL;) and remove the ORDER BY . Does it switch back to Hash Join? Why?

- **Observation:** Yes, it switched back to Hash Join (shown in Image 2).
- **Why the Optimizer prefers Hash Join:**
 - **Lower Total Cost:**
 - The **Merge Join** plan had an estimated total cost of 547.91 due to the heavy overhead of explicitly sorting 4,581 rows from inventory .
 - The **Hash Join** plan had an estimated total cost of only 193.39 .
 - **Faster Execution & Memory Efficiency:**
 - Hash Join builds an in-memory hash table on the smaller table (film , 1,000 rows, taking only 59kB of RAM) and probes it in linear time $O(N + M)$ without needing to sort any input.
 - Without an explicit ORDER BY requirement, sorting is unnecessary overhead, making Hash Join significantly faster and cheaper.

Hands on Lab

Task 1

Force a Sequential Scan by disabling Index Scans:

Query Query History

```
1 SET enable_indexscan = off;
2 SET enable_bitmapscan = off;
3
4 EXPLAIN (ANALYZE, BUFFERS)
5 SELECT * FROM rental WHERE rental_date > '2005-05-24';
```

Data Output Messages Notifications

Showing rows: 1 to 7

	QUERY PLAN	
	text	
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=10.870..26.334 rows=16044.00 loo...	
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)	
3	Buffers: shared read=150	
4	Planning:	
5	Buffers: shared hit=77 read=9	
6	Planning Time: 69.341 ms	
7	Execution Time: 26.928 ms	

Re-enable and compare the Cost and Time:

Query Query History

```
1 SET enable_indexscan = on;
2 SET enable_bitmapscan = on;
3
4 EXPLAIN (ANALYZE, BUFFERS)
5 SELECT * FROM rental WHERE rental_date > '2005-05-24';
```

Data Output Messages Notifications

Showing rows: 1 to

	QUERY PLAN	
	text	
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.012..1.393 rows=16044.00 loop...	
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)	
3	Buffers: shared hit=150	
4	Planning:	
5	Buffers: shared hit=3	
6	Planning Time: 0.179 ms	
7	Execution Time: 1.952 ms	

- **Optimizer's Choice:** Regardless of disabling or enabling index scans, PostgreSQL consistently chose a **Sequential Scan** (Seq Scan on rental) with the exact same cost estimate (cost=0.00..350.55) and expected rows (16044). This indicates that the filter rental_date

> '2005-05-24' retrieves such a massive portion of the table that the optimizer correctly deemed a full table scan cheaper than an index lookup.

- **Performance Drop:** The execution time decreased drastically from **26.928 ms** in the first run to **1.952 ms** in the second run. This improvement was entirely due to **Buffer Caching**; the first query had to perform disk reads (shared read=150), while the second query fetched data instantly from RAM (shared hit=150).

Task 2

Data Output		Messages	Notifications
                             			
Showing rows: 1 to 22			
	QUERY PLAN		
	text		
1	Hash Join (cost=150.55..545.57 rows=16044 width=20) (actual time=30.893..54.022 rows=16044.00 loops=1)		
2	Hash Cond: (r.inventory_id = i.inventory_id)		
3	Buffers: shared hit=177 read=7		
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=18) (actual time=28.671..42.035 rows=16044.00 loops=1)		
5	Hash Cond: (r.customer_id = c.customer_id)		
6	Buffers: shared hit=152 read=7		
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=10) (actual time=0.019..1.545 rows=16044.00 loops=1)		
8	Buffers: shared hit=150		
9	-> Hash (cost=14.99..14.99 rows=599 width=10) (actual time=28.640..28.642 rows=599.00 loops=1)		
10	Buckets: 1024 Batches: 1 Memory Usage: 33kB		
11	Buffers: shared hit=2 read=7		
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=10) (actual time=0.021..28.436 rows=599.00 loops=1)		
13	Buffers: shared hit=2 read=7		
14	-> Hash (cost=70.81..70.81 rows=4581 width=6) (actual time=2.175..2.176 rows=4581.00 loops=1)		
15	Buckets: 8192 Batches: 1 Memory Usage: 234kB		
16	Buffers: shared hit=25		
17	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.026..0.949 rows=4581.00 loops=1)		
18	Buffers: shared hit=25		
19	Planning:		
20	Buffers: shared hit=98 read=17 dirtied=1		
21	Planning Time: 129.177 ms		
22	Execution Time: 54.980 ms		
Total rows: 22		Query complete 00:00:00.234	

Challenge:

Data Output		Messages	Notifications
      			Showing rows: 1 to 22 
	QUERY PLAN text 		
1	Hash Join (cost=150.55..545.57 rows=16044 width=20) (actual time=1.117..8.530 rows=16044.00 loops=1)		
2	Hash Cond: (r.inventory_id = i.inventory_id)		
3	Buffers: shared hit=184		
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=18) (actual time=0.170..4.250 rows=16044.00 loops=1)		
5	Hash Cond: (r.customer_id = c.customer_id)		
6	Buffers: shared hit=159		
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=10) (actual time=0.016..0.820 rows=16044.00 loop...)		
8	Buffers: shared hit=150		
9	-> Hash (cost=14.99..14.99 rows=599 width=10) (actual time=0.149..0.168 rows=599.00 loops=1)		
10	Buckets: 1024 Batches: 1 Memory Usage: 33kB		
11	Buffers: shared hit=9		
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=10) (actual time=0.008..0.072 rows=599.00 loop...)		
13	Buffers: shared hit=9		
14	-> Hash (cost=70.81..70.81 rows=4581 width=6) (actual time=0.916..0.917 rows=4581.00 loops=1)		
15	Buckets: 8192 Batches: 1 Memory Usage: 234kB		
16	Buffers: shared hit=25		
17	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.007..0.372 rows=4581.00 loops=...)		
18	Buffers: shared hit=25		
19	Planning:		
20	Buffers: shared hit=56		
21	Planning Time: 0.466 ms		
22	Execution Time: 9.101 ms		
Total rows: 22		Query complete 00:00:00.070	

- **Join Strategy:** PostgreSQL executed a multi-level **Hash Join** across the three tables (`customer` , `rental` , `inventory`). Because there were no restrictive `WHERE` clauses, full table scans were required for all tables. Hash Join is the most memory-and-time efficient strategy for combining large, unsorted datasets.
- **Challenge (After `ANALYZE`):** Running the `ANALYZE` command **did not change** the execution plan algorithm; it remained a Hash Join. This proves that the optimizer's initial row statistics were already perfectly accurate. The execution time dropped from **54.980 ms** to

9.101 ms simply because the disk blocks were already warmed up in the memory cache during the previous run.